

Shielded Balance Tracking - Deep Dive

Purpose: Understand everything needed before implementing Phase 4B-Shielded

Prerequisites: You should already understand BIP-39/32, unshielded UTXOs, and the existing WebSocket subscription pattern

Table of Contents

1. [The Big Picture: Why Shielded is Different](#)
 2. [Key Hierarchy: Two Keys, Two Purposes](#)
 3. [The Shielded Coin: Anatomy of a Private Note](#)
 4. [Commitments and Nullifiers: The Twin Pillars](#)
 5. [The Merkle Tree: Proving Coins Exist](#)
 6. [Coin Encryption: How You Receive Coins Privately](#)
 7. [ZswapLocalState: The Wallet's Brain](#)
 8. [Sync Architecture: How Events Flow](#)
 9. [Balance Calculation: Available vs Pending](#)
 10. [The Full Coin Lifecycle](#)
 11. [What We Already Have](#)
 12. [What We Need to Build](#)
 13. [Comparison: Unshielded vs Shielded Patterns](#)
 14. [Key Decision: Rust-Backed State vs Kotlin State](#)
-

1. The Big Picture

Unshielded (What We Built)

With unshielded transactions, the blockchain is an open book. You subscribe to an address, and the indexer tells you: "Address X received 100 NIGHT in transaction Y." Simple.

```
Indexer says: "Address mn_addr_... received 100 NIGHT"
Wallet says: "Balance = 100 NIGHT"
```

Shielded (What We're Building)

With shielded transactions, **nobody can see who received what**. The blockchain stores encrypted blobs. Your wallet must:

1. Download ALL shielded events (not filtered by address)
2. Try to decrypt each one with your secret key
3. If decryption succeeds, that coin is yours
4. Track which coins you've spent (via nullifiers)
5. Calculate balance from what you can decrypt minus what you've spent

```
Indexer says: "Here are 500 encrypted events"
Wallet tries: decrypt(event_1) → fail, decrypt(event_2) → fail, ...
decrypt(event_47) → SUCCESS! 100 NIGHT for me!
Wallet says: "Balance = 100 NIGHT"
```

This is the fundamental difference. The indexer can't filter for you because it can't see who owns what. Privacy comes at the cost of your wallet doing more work.

2. Key Hierarchy: Two Keys, Two Purposes

From a single 32-byte seed (derived at BIP-32 path `m/44'/2400'/0'/3/0`), Midnight derives two independent key pairs:

```
Seed (32 bytes, from BIP-32 Zswap role)
├─ Coin Key Pair
│   ├── coin_secret_key (coin SK)
│   │   ├── Derived via: hash("midnight:csk" || seed)
│   │   └── Used for: SPENDING coins (generating nullifiers)
│   └── coin_public_key (coin PK)
│       ├── Derived via: hash("midnight:zswap-pk[v1]" || coin_sk_bytes)
│       └── Used for: IDENTIFYING you as the owner (part of your shielded address)
└─ Encryption Key Pair
    ├── encryption_secret_key (enc SK)
    │   ├── Derived via: sample_bytes("midnight:esk", seed, 64 bytes)
    │   │   └── This is a multi-round KDF (not a simple hash):
    │   │       ├── Round 0: SHA256("midnight:esk" || SHA256(0u64_le || seed))
    │   │       ├── Round 1: SHA256("midnight:esk" || SHA256(1u64_le || seed))
    │   │       └── Concatenate both 32-byte outputs → 64 bytes
    │   │           └── Then: Scalar::from_bytes_wide(64 bytes) → curve scalar
    │   └── Used for: DECRYPTING incoming coins
    └── encryption_public_key (enc PK)
        ├── Derived via: G * enc_sk (scalar multiplication on embedded curve)
        └── Used for: SENDERS encrypt coin data to you
```

Why Two Separate Key Pairs?

Separation of concerns:

- **Coin key** = spending authority. Whoever holds `coin_sk` can spend your money.

- **Encryption key** = receiving ability. Used to decrypt incoming notes so you can see your balance.

This separation enables interesting scenarios:

- A "view key" (enc_sk only) lets someone see your balance without spending ability
- The coin commitment binds to coin_pk (spending authority)
- The ciphertext is encrypted to enc_pk (receiving ability)

What We Already Have

Phase 1B already derives both public keys via JNI:

```
val shieldedKeys = ShieldedKeyDeriver.deriveKeys(zswapSeed)
// shieldedKeys.coinPublicKey      → "274c79e9..." (32 bytes hex)
// shieldedKeys.encryptionPublicKey → "f3ae706b..." (32 bytes hex)
```

What's missing: We only derive the public keys. For shielded balance tracking, we need the **secret keys** too (specifically encryption_secret_key for decrypting coins). This currently lives only in Rust.

3. The Shielded Coin: Anatomy of a Private Note

A shielded coin (also called a "note") is a simple data structure:

```
ShieldedCoinInfo {
  nonce: [u8; 32]           // Random, unique per coin (prevents linking)
  type:  [u8; 32]           // Token type identifier (NIGHT, DUST, etc.)
  value: u128               // Amount in atomic units
}
```

When a coin gets confirmed on-chain and placed in the Merkle tree, it becomes "qualified":

```
QualifiedShieldedCoinInfo extends ShieldedCoinInfo {
  mt_index: u64              // Position in the global Merkle tree
}
```

The mt_index is critical. You need it to create a Merkle proof that your coin exists. Without it, you can't spend. Think of it as the coin's "receipt number" from the blockchain.

What Lives On-Chain vs Off-Chain

Data	On-Chain?	Who Can See?
Commitment (hash of coin + owner)	Yes	Everyone (but it's a hash, reveals nothing)
Encrypted ciphertext	Yes	Everyone (but encrypted, unreadable)

Data	On-Chain?	Who Can See?
Coin value, type, nonce	No (encrypted)	Only the recipient (decrypts with enc_sk)
Nullifier	Yes (when spent)	Everyone (but can't link to commitment)

4. Commitments and Nullifiers: The Twin Pillars

These two concepts are the core innovation that makes shielded transactions work.

Commitment: "This coin exists"

```
commitment = hash("midnight:zswap-cc[v1]" || nonce || type || value ||
recipient_tag || recipient_key)
```

Where `recipient` is an enum:

- `User(coin_pk)` → tag = `true` (1 byte), key = coin public key (32 bytes)
- `Contract(address)` → tag = `false` (1 byte), key = contract address (32 bytes)

So the hash includes a boolean tag byte that distinguishes user-owned coins from contract-owned coins.

- A **one-way hash** of the coin's contents plus the tagged recipient identity
- Gets stored in a global Merkle tree on-chain
- Proves the coin exists without revealing its value, type, or owner
- Different recipients produce different commitments (same coin, different hash)

Analogy: A commitment is like a sealed envelope deposited in a vault. Everyone can see the envelope exists, but nobody can see what's inside.

Nullifier: "This coin has been spent"

```
nullifier = hash("midnight:zswap-cn[v1]" || nonce || type || value || sender_tag
|| sender_evidence)
```

Where `sender_evidence` is an enum:

- `User(coin_sk)` → tag = `true` (1 byte), evidence = coin secret key (32 bytes)
- `Contract(address)` → tag = `false` (1 byte), evidence = contract address (32 bytes)

This means both users AND contracts can produce nullifiers. The structure mirrors the commitment formula but uses the secret key instead of the public key.

- A **one-way hash** of the coin's contents plus the tagged sender evidence
- Revealed when spending (published on-chain)

- The blockchain checks: "Has this nullifier been seen before?" If yes, double-spend rejected.
- Only the coin owner can compute the User variant (requires `coin_sk`); contracts use their address

Analogy: A nullifier is like breaking the seal on the envelope. Once broken, everyone can see the seal is broken (spent), but they still can't see what was inside.

Why This Works (The Privacy Trick)

The key insight: **you cannot link a commitment to its nullifier** without knowing the secret key.

```
Observer sees:
Commitment C1 added to tree  (some coin was created)
Commitment C2 added to tree  (another coin was created)
...later...
Nullifier N7 published          (some coin was spent)

Observer CANNOT determine: Did N7 come from C1 or C2 or any other commitment?
Only the owner knows, because only they can compute: nullifier(coin, secret_key)
```

This is what makes Midnight private. The blockchain verifies correctness (via zero-knowledge proofs) without revealing the linkage.

5. The Merkle Tree: Proving Coins Exist

The blockchain maintains a **global Merkle tree** of all coin commitments ever created. This tree has a few important properties:

```
Merkle Tree (Append-Only)
├─ Slot 0: commitment_a    ← First coin ever created
├─ Slot 1: commitment_b
├─ Slot 2: commitment_c
├─ ...
└─ Slot N: commitment_z    ← Most recent coin
    ↑
    firstFree = N+1          ← Next available slot
```

Why the Wallet Needs a Local Copy

To spend a coin, you must prove it exists in the tree. This requires a **Merkle path** (the hashes from your coin's leaf up to the root).

The wallet maintains a local copy of the tree so it can:

1. Compute Merkle paths for spending
2. Know which `mt_index` each of your coins occupies
3. Fast-forward through sections it doesn't care about (via "collapsed updates")

What "firstFree" Means

`firstFree` is the next available slot in the tree. It tells the wallet:

- "The tree has grown to index N"
- "My local tree might be behind, I need to catch up"
- Used for Merkle proof generation and sync gap detection

6. Coin Encryption: How You Receive Coins Privately

When someone sends you shielded coins, the transaction output contains:

```
Output (zswap/src/structure.rs) {
  coin_com:      Commitment           // Goes in Merkle tree
  value_commitment: Pedersen          // Pedersen commitment to the
value
  contract_address: Option<ContractAddress> // Set for contract-owned
coins
  ciphertext:     Option<CoinCiphertext> // Encrypted coin data
(optional!)
  proof:          ZK proof that everything is valid
}
```

Note: `ciphertext` is **optional**. When the output's preimage evidence is a `PublicPreimage` (unencrypted coin info + recipient, used for contract-owned outputs), there's no ciphertext.

The `ZswapPreimageEvidence` enum has three variants:

- `Ciphertext (CoinCiphertext)` - encrypted, needs trial decryption
- `PublicPreimage { coin, recipient }` - plaintext, checked via public key match
- `None` - no evidence (certain contract scenarios)

The Encryption Scheme (El Gamal + CTR mode)

Midnight uses a SNARK-friendly encryption scheme based on El Gamal key agreement with CTR mode using `transient_hash` as a block cipher. Field addition replaces XOR (since field operations are native to the ZK circuit).

```
Sender's side (transient-crypto/src/encryption.rs):
1. Generate random ephemeral secret y
2. Compute ephemeral_pk = G * y           (curve point g^y)
3. Compute K* = recipient_enc_pk * y     (El Gamal: g^(xy))
4. Derive key K = transient_hash(K*.x, K*.y) (hash both coordinates)
5. Prepend a zero element to CoinInfo fields (integrity check)
6. Encrypt via CTR: ciph[i] = transient_hash(K, i) + plaintext[i]
7. Output: CoinCiphertext = (ephemeral_pk, 6 encrypted field elements)
```

The 6 field elements come from:

- 1 zero element (integrity check, prepended)
- 2 fields for nonce (HashOutput = 32 bytes = 2 field elements)

- 2 fields for type (HashOutput = 32 bytes = 2 field elements)
- 1 field for value (u128)

Recipient's side (YOUR wallet):

1. Compute $K^* = \text{ephemeral_pk} * \text{your_enc_sk}$ (same shared secret: $g^{(xy)}$)
2. Derive same key $K = \text{transient_hash}(K^*.x, K^*.y)$
3. Decrypt: $\text{plaintext}[i] = \text{ciph}[i] - \text{transient_hash}(K, i)$
4. Check: $\text{plaintext}[0] == 0?$ (integrity check)
 - If NOT zero → decryption failed, this coin is not ours
 - If zero → SUCCESS, extract CoinInfo from remaining 5 fields

Trial Decryption: The Discovery Process

Here's where it gets interesting. When your wallet syncs, for each output event:

For each ZswapOutput event:

1. Check: Is commitment in my pending_outputs? (expected self-transfer)
 - If yes → Coin is mine! Add to coins. Done.
2. Check: Is preimage a PublicPreimage with my coin_pk as recipient?
 - If yes → Coin is mine! Add to coins. Done.
3. Try: decrypt(output.ciphertext, my_enc_sk) (trial decryption)
 - If success → Coin is mine! Add to coins. Done.
 - If failure → Not my coin. Collapse Merkle branch. Skip.

Step 3 is **trial decryption** - the most common path. Your wallet tries to decrypt EVERY encrypted output on the blockchain. Only outputs encrypted to your `enc_pk` will succeed.

Performance implication: Your wallet processes every shielded transaction on the network, not just your own. This is why Midnight handles this in Rust (WASM in TypeScript, native FFI for us) - it needs to be fast.

7. ZswapLocalState: The Wallet's Brain

The TypeScript SDK wraps all shielded state in a class called `ZswapLocalState`. This is the single most important data structure for shielded balance tracking:

```
ZswapLocalState (Rust internal structure) {
  coins: Map<Nullifier → QualifiedCoinInfo>
    // Confirmed, spendable coins you own, keyed by their nullifier
    // Each value has: nonce, type, value, mt_index
    // Note: TypeScript WASM bindings expose this as
  Set<QualifiedShieldedCoinInfo>,
    // but internally it's a Map so the wallet can quickly look up coins by
  nullifier
    // when processing spent events

  pending_outputs: Map<Commitment → CoinInfo>
    // Coins you expect to receive but haven't confirmed yet
    // Example: You sent coins to yourself, waiting for confirmation
    // Note: No TTL field stored (the WASM bindings show TTL as undefined
  placeholder)
```

```

pending_spends: Map<Nullifier → QualifiedCoinInfo>
    // Coins you've initiated spending but haven't finalized
    // Prevents double-spending your own coins
    // Note: No TTL field stored (same as pending_outputs)

merkle_tree: MerkleTree
    // Local copy of the global commitment tree
    // Needed for generating spending proofs
    // Not exposed in TypeScript API, only accessible internally in Rust

first_free: u64
    // Next available Merkle tree slot
    // Tells you where the tree ends
}

```

Key Methods on ZswapLocalState

```

// Create empty state (new wallet)
new ZswapLocalState()

// === CORE METHODS (exist in Rust core) ===

// Sync: Process blockchain events (THE MAIN METHOD for balance tracking)
// Defined via ZswapLocalStateExt trait in ledger/src/semantics.rs
state.replayEvents(secretKeys, events) → ZswapLocalState

// Sync with change tracking (tells you what coins were received/spent)
state.replayEventsWithChanges(secretKeys, events) → ZswapLocalStateWithChanges

// Spend a coin (creates a ZswapInput for transaction building)
// Note: ttl param exists in WASM bindings but is unused in Rust core
state.spend(secretKeys, coin, segment) → [newState, UnprovenInput]

// Spend directly into an output (for transient coins)
state.spendFromOutput(secretKeys, coin, segment, output) → [newState, Transient]

// Apply a transaction offer to the state
state.apply(secretKeys, offer) → ZswapLocalState

// Handle failed transactions (rollback pending state)
state.applyFailed(failedTx) → ZswapLocalState

// Watch for expected coin (self-transfers, contract outputs)
state.watchFor(coinPublicKey, coinInfo) → ZswapLocalState

// Skip Merkle tree sections (fast-forward through gaps)
state.applyCollapsedUpdate(update) → ZswapLocalState

// Persistence
state.serialize() → Uint8Array
ZswapLocalState.deserialize(bytes) → ZswapLocalState

// === WASM-ONLY (not in Rust core) ===

// clearPending exists only in WASM bindings and is a no-op (returns clone of

```

```
state)
// state.clearPending(time) → ZswapLocalState // DO NOT rely on this
```

The Critical Insight

`replayEvents` does ALL the heavy lifting. It receives already-deserialized `Event` objects (deserialization from hex bytes happens upstream in the SDK/FFI layer) and processes them:

1. For each `ZswapOutput` event:
 - a. Updates Merkle tree with the commitment
 - b. Checks if commitment is in `pending_outputs` (expected coin from self-transfer)
 - c. If not, trial-decrypts the ciphertext with your `enc_sk`
 - d. On success: computes nullifier, adds coin to `coins` map (keyed by nullifier)
 - e. On failure: collapses the Merkle tree branch (space-efficient skip)
2. For each `ZswapInput` event (nullifier published):
 - a. Removes nullifier from `coins` map (coin is spent)
 - b. Removes nullifier from `pending_spends` map (spend is finalized)
3. Skips all non-zswap events (dust, contract deploy, etc.)
4. Returns the updated state

We don't need to reimplement any of this. The Rust implementation already exists. We just need to expose it via FFI, similar to how we already handle `DustLocalState`.

8. Sync Architecture: How Events Flow

The GraphQL Subscription

The TypeScript SDK subscribes to `ZswapEvents`:

```
subscription ZswapEvents($id: Int) {
  zswapLedgerEvents(id: $id) {
    id      # Sequential event number (for resume)
    raw     # Hex-encoded serialized ledger.Event
    maxId   # Highest event the indexer has seen
  }
}
```

Key differences from unshielded subscription:

Aspect	Unshielded	Shielded
Filter	By address	None (get ALL events)
Data format	Parsed JSON (values visible)	Raw hex bytes (serialized, encrypted)
Processing	Simple field mapping	Deserialize + trial decrypt in Rust

Aspect	Unshielded	Shielded
Volume	Only your transactions	ALL shielded transactions

The Sync Flow

```

1. Subscribe to zswapLedgerEvents(id: lastProcessedId)
   ↓
2. Receive batch of events: [{id: 42, raw: "0a1b2c...", maxId: 100}, ...]
   ↓
3. Pass raw hex to Rust: state.replayEvents(secretKeys, events)
   ↓
4. Rust internally:
   a. Deserialize each event from bytes
   b. Trial-decrypt outputs with enc_sk
   c. Add discovered coins to state
   d. Process nullifiers (mark spent coins)
   e. Update Merkle tree
   ↓
5. Get back: updated ZswapLocalState + changes (received/spent coins)
   ↓
6. Kotlin: Update database, emit balance via Flow
   ↓
7. UI: Display updated shielded balance

```

Batching and Performance

The TypeScript SDK batches events in two stages:

- Batch collection:** `Stream.groupedWithin(batchSize, Duration.millis(1))` - collects up to 10 events (default) within a 1ms window, emitting whichever limit is hit first
- Emission spacing:** `Schedule.spaced(Duration.millis(4))` - waits 4ms between emitting successive batches

This two-stage approach is important because:

- Trial decryption is CPU-intensive
- The 1ms collection window groups rapid events efficiently
- The 4ms emission spacing prevents overwhelming the consumer
- Together they provide smooth sync progress without blocking the UI

9. Balance Calculation: Available vs Pending

Three Balance Types

```

Available Balance = SUM(coins.values) - SUM(pendingSpends.values)
→ Coins you can spend RIGHT NOW

```

```
Pending Incoming = SUM(pendingOutputs.values)
    → Coins you expect to receive but haven't been confirmed

Pending Outgoing = SUM(pendingSpends.values)
    → Coins you're in the process of spending

Total Balance = Available + Pending Incoming
```

Reading Balance from ZswapLocalState

```
In Rust, coins is a Map<Nullifier, QualifiedCoinInfo>, so available coins
are those whose nullifier key is NOT also present in pending_spends.

// Available coins (confirmed, not being spent)
available_coins = coins.entries
    .filter(|(nullifier, _)| !pending_spends.contains_key(nullifier))
available_balance = available_coins.sum(|(_, coin)| coin.value)

// Pending incoming
pending_incoming = pending_outputs.values().sum(|coin| coin.value)

// Pending outgoing
pending_outgoing = pending_spends.values().sum(|coin| coin.value)
```

Since both `coins` and `pending_spends` are keyed by `Nullifier`, checking overlap is a simple key lookup - no need to recompute nullifiers.

Token Type Grouping

Coins can be different token types (NIGHT, contract tokens, etc.). Balance should be grouped:

```
Shielded Balance:
NIGHT: 500,000 available + 100,000 pending
TOKEN_X: 1,000 available
```

10. The Full Coin Lifecycle

```
1. CREATION
    Someone builds a transaction with an output for you:
    - CoinInfo created (random nonce, type, value)
    - Commitment computed: hash(domain || coin || your_cpk)
    - Ciphertext created: encrypt(coinInfo, your_enc_pk)
```

↓

2. ON-CHAIN

Transaction submitted **to** Midnight network:

- Commitment added **to global** Merkle tree at index N
- Ciphertext stored alongside
- **Event** emitted **by** the ledger



3. DISCOVERY (Your wallet syncs)

Your wallet receives the **event**:

- Downloads raw hex bytes **from** indexer
- Deserializes hex → **Event** objects (SDK/FFI layer)
- Calls `replayEvents(secretKeys, [event])`
- Rust tries: `decrypt(ciphertext, enc_sk) → SUCCESS`
- Extracts: nonce, type, value
- Creates `QualifiedCoinInfo` **with** `mt_index = N`
- Adds **to** coins **set**
- Updates local Merkle tree



4. AVAILABLE

Coin sits **in** your wallet's *coin set*:

- Contributes **to** available balance
- Can be selected **for** spending
- Has a Merkle path (can prove existence)



5. SPENDING (When you want to send)

`state.spend(secretKeys, coin, segment)` returns:

- UnprovenInput **with** nullifier
- Updated state: coin COPIED **to** `pending_spends`
(coin stays **in** coins map too - **not** removed!)
- Nullifier: `hash(domain || coin || User(coin_sk))`
- Balance: `coins - pending_spends` (subtraction needed)



6. CONFIRMATION

Transaction finalized **on-chain**:

- Nullifier recorded **in global** nullifier **set**
- **Next** `replayEvents()` sees the `ZswapInput` **event**
- Coin removed **from** BOTH coins **AND** `pending_spends`
- Coin **is** gone forever (**double**-spend impossible)

If transaction FAILS:

- `applyFailed()` removes coin **from** `pending_spends` only
- Coin remains **in** coins map → back **to** available

11. What We Already Have

Shielded Keys (Phase 1B) - COMPLETE

```
rust/kaira-crypto-ffi/src/lib.rs
  → derive_shielded_keys(seed) → (coin_pk, enc_pk)

core/crypto/src/main/kotlin/.../shielded/
  → ShieldedKeyDeriver.kt   (JNI bridge)
  → ShieldedKeys.kt         (data class)
  → MemoryUtils.kt          (secure wipe)
```

We derive **public keys** only. For Phase 4B-Shielded we'll need secret keys too (for `replayEvents`).

Dust State Pattern (Phase 2-DUST) - REUSABLE PATTERN

The dust implementation already shows us exactly how to handle Rust-backed state in Android:

```
rust/kaira-crypto-ffi/src/dust_ffi.rs
  → create_dust_local_state()
  → dust_replay_events(state, seed, events_hex) → new_state
  → dust_wallet_balance(state, time) → decimal_string
  → serialize_dust_state(state) → bytes
  → deserialize_dust_state(bytes) → state

core/crypto/src/main/kotlin/.../dust/DustLocalState.kt
  → JNI wrapper managing native pointer lifecycle
  → serialize()/deserialize() for persistence
  → replayEvents() for syncing
  → getBalance() for balance queries
```

This is almost exactly what we need for shielded. The pattern:

1. Rust holds the state (pointer)
2. Kotlin wraps it (JNI lifecycle management)
3. Sync via `replayEvents()` with raw blockchain data
4. Query balance from Rust
5. Serialize/deserialize for database persistence

WebSocket Client (Phase 4B-1) - REUSABLE

```
core/indexer/src/main/kotlin/.../websocket/
  → GraphQLWebSocketClient.kt (already works)
  → Just need to add a new subscription query
```

Unshielded Balance Pattern (Phase 4B) - REFERENCE ARCHITECTURE

```
Subscription → UtxoManager → Database → BalanceRepository → ViewModel → UI
```

Same flow, different data source.

12. What We Need to Build

New Rust FFI Functions

```
// Create new empty shielded state
create_zswap_local_state() → *mut ZswapLocalState

// The main sync method - process blockchain events
zswap_replay_events(
    state: *mut ZswapLocalState,
    seed: *const u8,           // 32-byte zswap seed (derives secret keys
internally)
    events_hex: *const c_char // JSON array of hex-encoded events
) → *mut ZswapLocalState     // Returns new state pointer

// Query balance by token type
zswap_get_balance(
    state: *const ZswapLocalState
) → *const c_char           // JSON: {"NIGHT": "500000", "TOKEN_X": "1000"}

// Get coin count
zswap_get_coin_count(state: *const ZswapLocalState) → u32

// Persistence
serialize_zswap_state(state: *const ZswapLocalState) → (*const u8, usize)
deserialize_zswap_state(data: *const u8, len: usize) → *mut ZswapLocalState

// Cleanup
free_zswap_local_state(state: *mut ZswapLocalState)
```

New Kotlin Classes

```
// 1. JNI wrapper (modeled after DustLocalState)
class ZswapLocalState {
    private var nativePtr: Long // Rust pointer

    fun replayEvents(seed: ByteArray, eventsHex: List<String>): ZswapLocalState
    fun getBalances(): Map<String, BigInteger>
    fun getCoinCount(): Int
    fun serialize(): ByteArray
```

```

        companion object {
            fun create(): ZswapLocalState
            fun deserialize(data: ByteArray): ZswapLocalState
        }
    }

    // 2. GraphQL subscription query
    // In GraphQLQueries.kt:
    const val ZSWAP_EVENTS_SUBSCRIPTION = """
        subscription ZswapEvents(${'$'}id: Int) {
            zswapLedgerEvents(id: ${'$'}id) {
                id
                raw
                maxId
            }
        }
    """

    // 3. Shielded balance manager
    class ShieldedBalanceManager {
        fun startSync(seed: ByteArray)
        fun observeBalances(): Flow<Map<String, BigInteger>>
        fun observeSyncProgress(): Flow<SyncProgress>
    }

    // 4. Database entity (minimal - just for persistence)
    @Entity(tableName = "shielded_state")
    data class ShieldedStateEntity(
        @PrimaryKey val id: Int = 0, // Singleton
        val serializedState: ByteArray,
        val lastEventId: Long,
        val lastUpdated: Long
    )

```

GraphQL Subscription Differences

```

Unshielded (existing):
    subscription UnshieldedTransactions($address: UnshieldedAddress!,
$transactionId: Int) {
        unshieldedTransactions(address: $address, transactionId: $transactionId) {
            ... parsed JSON with values, addresses, etc.
        }
    }
→ Filtered by address
→ Returns parsed, readable data

Shielded (new):
    subscription ZswapEvents($id: Int) {
        zswapLedgerEvents(id: $id) {
            id // Event sequence number
            raw // Opaque hex bytes (MUST process in Rust)
            maxId // Progress tracking
        }
    }

```

→ NOT filtered (gets ALL shielded events)
→ Returns raw bytes (Rust deserializes + decrypts)

13. Comparison: Unshielded vs Shielded Patterns

Aspect	Unshielded (Phase 4B)	Shielded (Phase 4B-Shielded)
Subscription	<code>unshieldedTransactions(address)</code>	<code>zswapLedgerEvents(id)</code>
Filtering	Server-side (by address)	Client-side (trial decryption)
Data format	Parsed JSON	Raw hex bytes
Processing	Kotlin (simple field mapping)	Rust (deserialize + decrypt)
State tracking	Room database (UTXOs)	Rust ZswapLocalState (serialized to DB)
Balance calc	SUM(available UTXOs)	Query from Rust state
Coin selection	Kotlin UtxoSelector	Rust state.spend()
Double-spend prevention	DB state (AVAILABLE/PENDING/SPENT)	Rust pendingSpends map
Resume sync	<code>transactionId</code> parameter	<code>id</code> parameter (event index)
Privacy	None (address visible)	Full (trial decryption)
Volume	Only your transactions	ALL shielded transactions

14. Key Decision: Rust-Backed State vs Kotlin State

Option A: Rust-Backed State (Recommended)

Let Rust's `ZswapLocalState` handle everything, expose via FFI.

Pros:

- Exact same logic as TypeScript SDK (uses same Rust code via WASM)
- No risk of crypto implementation bugs
- Trial decryption, Merkle tree, nullifier tracking all handled
- Already proven to work (DustLocalState pattern)
- `replayEvents()` does all the heavy lifting in one call

Cons:

- Larger APK (already accepted this trade-off in Phase 1B)
- JNI boilerplate (we have the pattern from dust)
- State is opaque (can only query via FFI, not inspect directly)

Option B: Kotlin State

Reimplement balance tracking in pure Kotlin.

Pros:

- More debuggable (can inspect state directly)
- No JNI complexity

Cons:

- Must reimplement: trial decryption, Merkle tree, nullifier matching
- Must reimplement: all the crypto (JubJub curve operations, ECDH)
- High risk of bugs in security-critical code
- Massive effort (weeks, not hours)

Recommendation: Option A

Follow the DustLocalState pattern exactly. The Rust `ZswapLocalState` has `replayEvents()` which does everything we need. We just need to:

1. Expose it via FFI
2. Wrap in Kotlin with JNI lifecycle management
3. Serialize/deserialize for database persistence
4. Query balances and emit via Flow

This keeps Phase 4B-Shielded in the 8-12h estimate range.

Summary: What You Need to Know Before Starting

1. **Two key pairs** from one seed: coin key (spending) + encryption key (receiving)
 2. **Commitments** prove coins exist (stored in Merkle tree on-chain)
 3. **Nullifiers** prove coins are spent (stored in nullifier set on-chain)
 4. **Trial decryption** is how you discover your coins (try to decrypt every output)
 5. **ZswapLocalState** tracks everything: coins, pending, Merkle tree
 6. **replayEvents()** is the single method that handles all sync logic
 7. **The DustLocalState pattern** is your implementation blueprint
 8. **Raw hex events** come from the indexer, Rust processes them entirely
 9. **Balance = coins not in pending_spends** (coins and pending_spends share nullifier keys)
 10. **Serialize/deserialize** for persistence, just like dust state
-

Source References

All claims in this document have been verified against the official Midnight libraries.

TypeScript SDK (midnight-wallet):

- `packages/shielded-wallet/src/v1/CoreWallet.ts` - State management, CoinHashesMap
- `packages/shielded-wallet/src/v1/Sync.ts` - Event sync architecture, SecretKeysResource, batching
- `packages/indexer-client/src/graphql/subscriptions/ZswapEvents.ts` - GraphQL subscription
- `packages/hd/src/HDWallet.ts` - BIP-32 path, Roles enum (Zswap = 3), coin type 2400

Rust (midnight-ledger):

- `zswap/src/keys.rs` - Key derivation (coin SK, enc SK, sample_bytes KDF)
- `zswap/src/local.rs` - State struct (coins Map, pending_spends, pending_outputs, merkle_tree)
- `zswap/src/structure.rs` - CoinCiphertext (COIN_CIPHERTEXT_LEN = 6), Input/Output
- `coin-structure/src/coin.rs` - CoinInfo, QualifiedInfo, commitment(), nullifier(), Recipient/SenderEvidence enums
- `coin-structure/src/transfer.rs` - SenderEvidence enum definition
- `transient-crypto/src/encryption.rs` - El Gamal encryption, CTR mode, transient_hash
- `ledger/src/semantics.rs` - ZswapLocalStateExt trait (replay_events, replay_events_with_changes)
- `ledger/src/events.rs` - ZswapPreimageEvidence, try_with_keys (trial decryption entry point)
- `ledger-wasm/src/zswap_state.rs` - WASM bindings (TTL fields are undefined placeholders)

Existing Kuira Code:

- `rust/kuira-crypto-ffi/src/lib.rs` - Shielded key derivation FFI
- `rust/kuira-crypto-ffi/src/dust_ffi.rs` - DustLocalState FFI (BLUEPRINT)
- `core/crypto/src/main/kotlin/.../dust/DustLocalState.kt` - Kotlin JNI wrapper (BLUEPRINT)
- `core/crypto/src/main/kotlin/.../shielded/ShieldedKeyDeriver.kt` - Existing shielded key bridge